专业课程设计 II 报告

（ 2026 / 2027 学年　第 一 学期 ）

题　　目： 知识管理系统

专　　业： 【专业】

学生姓名： 【姓名】

班级学号： 【班级】 【学号】

指导教师： 朱　曼

指导单位： 计算机科学与技术系

日　　期： 2026.10.12 – 2026.10.23

专业课程设计 II 能力测评表

课程目标	评价准则	计分（每项 10 分）
课程目标 1：通过课程设计提高学生的理论联系实际、查阅资料、自学和独立分析与解决复杂工程问题的能力（30 分）	1、能够掌握计算机软件设计的相关基础知识，并能够针对求解的工程问题，进行合理的分析与设计。 2、能够给出数据结构和算法的设计描述，给出关键算法的流程图或伪代码，并给出各算法之间的结构关系描述。 3、具备一定的人机交互设计意识，人机交互设计合理、友好，操作简便。	
课程目标 2：要求学生在深入理解软件系统的相关知识基础上，综合运用所学的方法和理论自主地完成课程设计。要求学生通过软件课题的开发实践活动熟悉一种主流的数据库管理系统产品和软件开发工具，掌握软件设计和开发的理论方法与实施步骤。要求学生在软件开发的过程中考虑到可持续发展的因素，例如安全以及环保等。（30 分）	4、能够结合计算机软硬件资源，合理选用算法、数据结构、数据存储方式等技术手段，对求解的工程问题进行有效建模和求解。 5、能够选择合适的程序设计语言与软件开发平台，对求解的工程问题进行编程实现。掌握调试方法与工具，对程序开发过程中出现的问题进行分析、跟踪与调试，并能够进行充分测试。 6、在完成课题要求的各项任务和指标时能够充分考虑可持续发展因素，例如安全和环保等。	
课程目标 3：要求学生完成一次项目设计与开发的全过程，掌握软件需求分析、软件设计、软件测试和团队协同开发的基本理论和技术实践。应用工程项目管理原理和经济决策方法划分和管理课题任务，培养项目组织管理能力。（20 分）	7、能够正确理解课题目标任务，并做好需求分析、软件设计、软件测试等关键环节的实践工作，具备团队协调、协作和组织管理能力。 8、具备一定的语言表达能力与文字处理能力，能够结合复杂工程问题撰写报告，报告内容和实验数据详实，格式规范。	
课程目标 4：要求学生能够通过参与项目设计与开发的全过程，建立解决实际问题需要不断探索和学习的主动性意识，能够结合实际问题的特征需求，通过自主学习、终生学习，拓展自身的知识和能力。（20 分）	9、具备一定自学能力与探索创新意识，能够充分利用教科书及其资源（如网络等）自学新知识与新技能，以解决实际问题。 10、能够正确、完整地回答指导教师关于课题开发全过程的问询，能够掌握解决具体问题的一般性规律和方法，具备自主学习、终生学习的意识。	

专业课程设计 II 能力测评总分：

指导教师：　　2026 年 12 月 31 日

备注：

知识管理系统

一、课题内容和要求

1.1 课题背景

知识管理系统（Knowledge Management System，简称 KMS）是一类帮助组织与个人有效地创建、分享、应用和维护知识的软件系统。它通过技术手段与组织文化的结合促进知识的流动与复用，从而提升组织的创新能力与竞争力。对于高校课程教学而言，一个学期内产生的讲义、实验指导、技术报告、参考论文与教学视频等资料数量可观，但这些资料往往散落在网盘、聊天记录与个人电脑中：学生找不到、教师不便复用、下一届学生几乎从零开始。

造成这种状况的根本原因不只是“没有地方存”，而是“存了也找不到”与“找到了也不知道它与其他知识有什么关系”。前者是检索问题，后者是知识组织问题。本课题正是围绕这两个问题展开：一方面建立结构化的知识库与支持中文的高效检索；另一方面用知识本体把零散的资料组织成一张具有语义关系的网络，并在此基础上提供基于检索增强生成（RAG）的智能问答。

1.2 课题任务描述

依据《专业课程设计 II 任务书》题目一的要求，本课题需要为《专业课程设计 II》课程搭建一套知识管理系统，具体任务包括以下四个部分。

（1）知识库：存储与《专业课程设计 II》相关的各种知识资产，如技术报告、文档、论文、视频等；允许用户搜索、浏览、分类和归档知识内容；应注重知识的内在结构（如层次关系），用本体对知识的逻辑结构进行建模，本体需覆盖至少 20 个知识点、5 种关系。

（2）搜索引擎：快速定位知识库中的相关内容，支持全文搜索、高级搜索和 RAG 问答功能。

（3）内容管理系统（CMS）：用于创建、编辑、发布和管理知识内容；支持多种文件格式和多媒体内容；需提供指定天数/修改次数的版本管理。

（4）系统界面美观，使用友好。

1.3 完成情况概述

本课题完整实现了任务书要求的全部功能，并在本体规模、检索质量与工程可靠性三个方面做了额外的工作。系统采用前后端分离架构，后端为 Spring Boot 3.5 + Java 21 + MySQL 8.0，前端为 Vue 3 + Element Plus + ECharts；共实现 98 个 Java 源文件、26 个 Vue 组件、20 张数据库表与 60 余个 REST 接口。系统实际达到的规模如表 1 所示。

项目	实际规模	任务书要求	说明
知识点（本体概念）	107 个	≥ 20 个	覆盖软件工程、数据库、知识管理三个知识域
本体关系类型	7 种	≥ 5 种	上下位、组成、先修、相关、应用于、产出、近义
本体关系实例	183 条	未作要求	其中 104 条由层次结构自动生成，79 条人工标注
知识条目	60 条	未作要求	含文档、技术报告、论文、视频等 8 种资产类型
知识分类	18 个	未作要求	三层树形结构
知识标签	30 个	未作要求	扁平的交叉分类维度
多媒体附件	11 个	支持多格式	PDF / PNG / DOCX / XLSX / Markdown
检索分块	164 个	未作要求	供 RAG 混合检索召回
本体覆盖率	93.46%	未作要求	107 个知识点中已有 100 个被知识条目引用

表 1 系统实际实现规模

在上述功能之外，系统还实现了本体推理（沿关系推导学习路径）、实体链接（从文本自动识别知识点）、检索结果重排序、版本差异比对与回滚、审计日志与统计看板等辅助功能。这些功能使得系统不只是一个“能存能搜”的资料库，而是一个可以回答“知识之间是什么关系”的平台。

二、需求分析和总体设计

2.1 本课题的主要功能包括

通过对课程教学场景的分析，本系统识别出三类使用者，并按最小权限原则划分角色。角色的划分依据是“是否需要修改知识库内容”而不是身份，因为浏览者的高频操作是检索与提问，而编辑与管理员的高频操作是内容维护。

角色	定位	主要功能
浏览者	学生、旁听者	检索知识、浏览知识图谱、向知识库提问、下载附件
编辑	课程助教、资料整理者	创建与编辑条目、提交审核并发布、管理版本与回滚、归档与恢复
管理员	课程负责人	用户与角色管理、维护知识本体、配置版本策略、重建检索索引、查看审计日志

表 2 系统角色与功能划分

系统的功能可以归纳为五个核心模块：知识库模块负责知识资产的存储与组织；知识本体模块负责用概念与语义关系建模知识的逻辑结构；搜索引擎模块负责全文检索与高级检索；检索增强问答模块负责基于知识库的自然语言问答；内容管理模块负责内容的生命周期与版本管理。此外还有归档、审计与统计看板三个支撑模块。各模块与三类使用者的关系如图 1 所示。

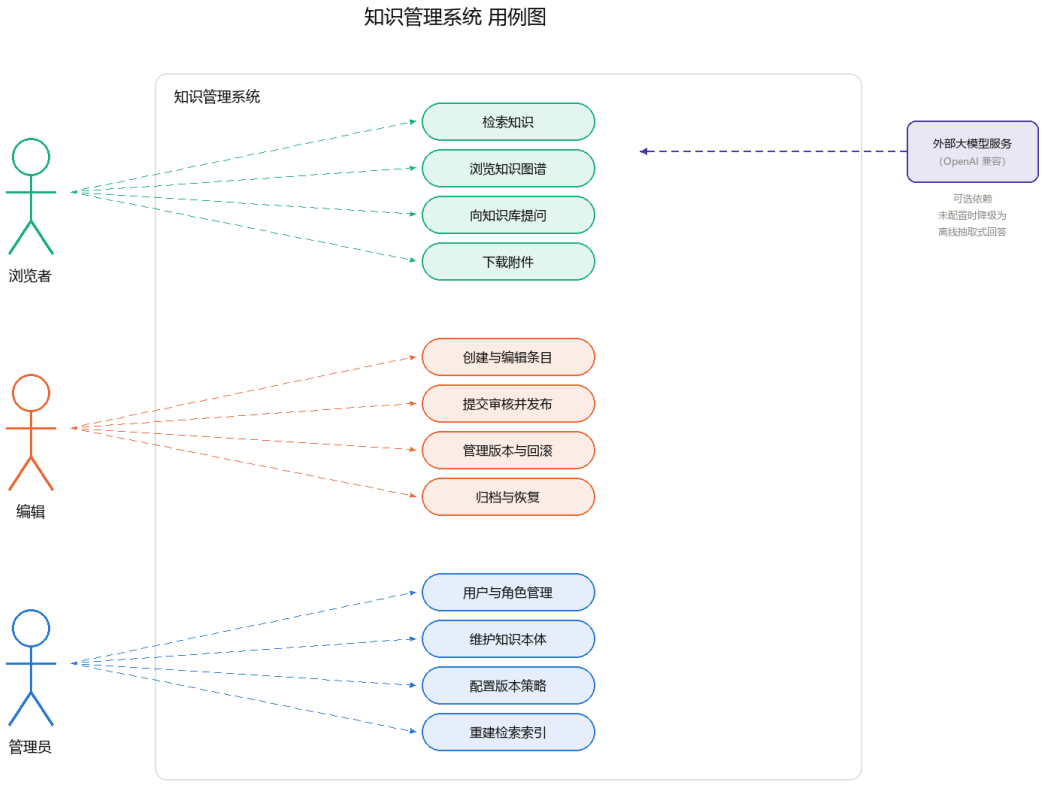


图 1 知识管理系统用例图

图 1 知识管理系统用例图

从图 1 可以看出，三个角色与系统的交互是层层递进的：浏览者只做只读操作；编辑在只读基础上增加了内容写入与版本管理；管理员则负责系统的配置与知识结构本身。右下角的“外部大模型服务”是一个可选依赖 —— 系统的智能问答在未配置大模型接口时会降级为离线抽取式回答，这一点将在 3.2 节展开说明。

知识条目是系统中生命周期最长的对象，它需要在“可以自由修改”与“对外可见的内容必须经过确认”这两个诉求之间取得平衡。为此系统为知识条目设计了四态流转模型，如图 2 所示。

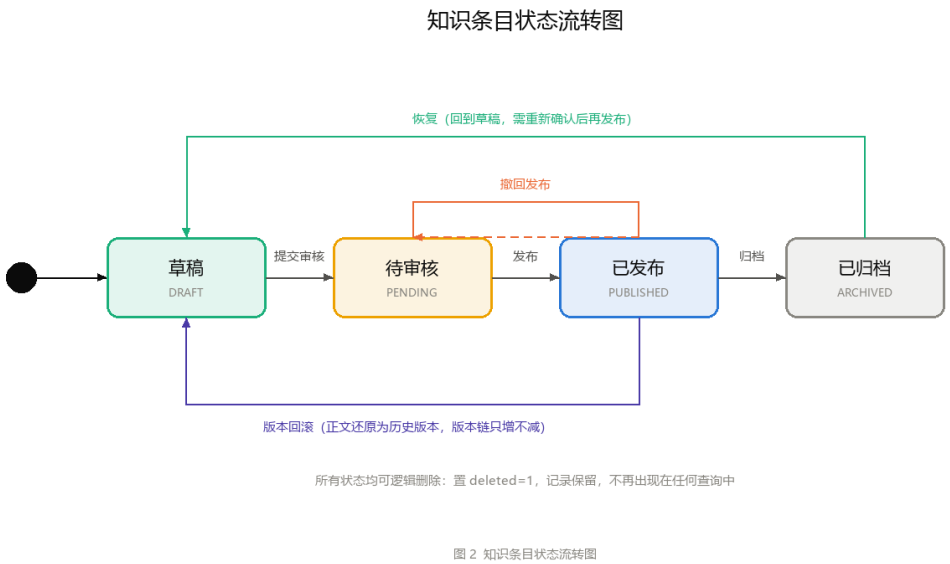


图 2 知识条目状态流转图

图 2 知识条目状态流转图

状态模型中有三个设计决策需要说明。第一，只有“已发布”状态的条目会进入公开检索与问答的召回范围，草稿与待审核的条目仅作者与管理员可见，避免未完成的内容被误当作结论引用。第二，归档不是删除：正文、附件与版本历史全部保留，只是移出常规检索范围，并且归档原因会被记录下来 —— “为什么不再推荐这份资料”本身也是有价值的信息。第三，从归档恢复后条目回到“草稿”而不是直接回到“已发布”，因为归档通常意味着内容可能已经过时，直接恢复为已发布会让未经复核的内容对所有人可见。

智能问答是本系统中流程最长的功能，它跨越了本体、检索与生成三个模块，其完整的调用时序如图 3 所示。

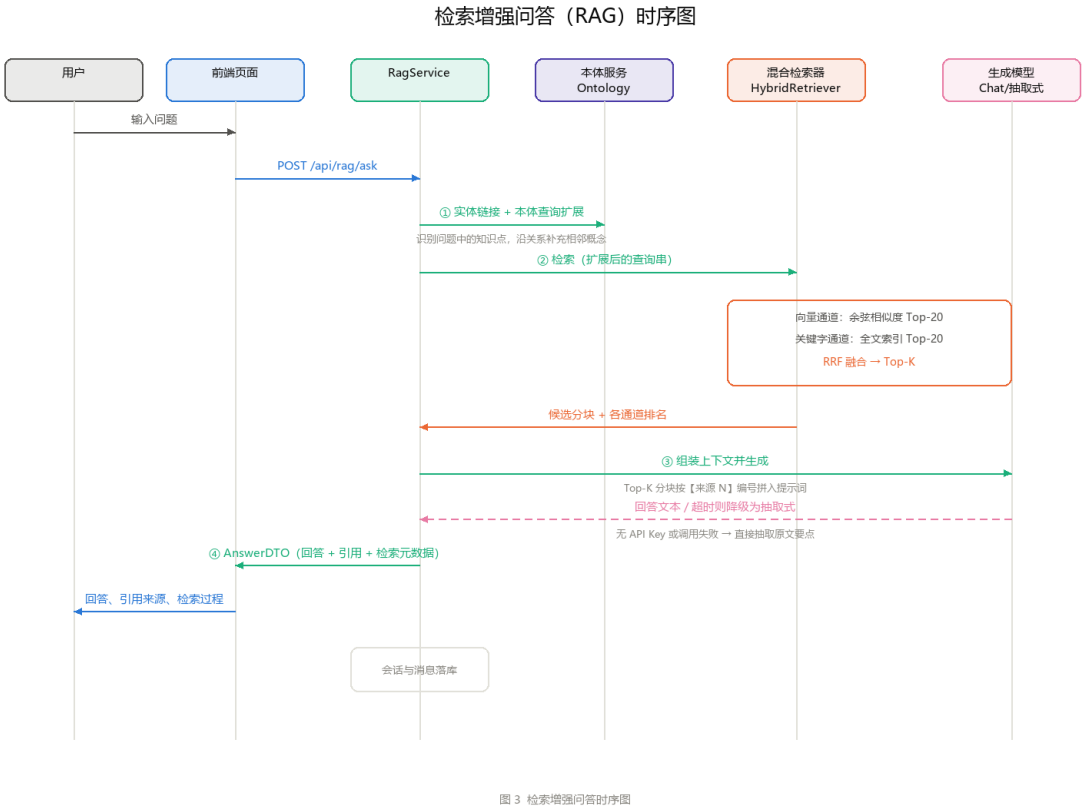


图 3 检索增强问答时序图

图 3 检索增强问答时序图

从图 3 可以看到，一次问答在服务端要经历五个步骤：本体查询扩展、双通道混合检索、RRF 融合、上下文组装与生成、以及会话落库。其中“本体查询扩展”是本系统区别于通用 RAG 实现的一处改进：一般的 RAG 直接用用户原话去检索，而本系统先用实体链接识别出问题中的知识点，再沿本体的先修、组成、相关等关系补充相邻概念，把这些词一并拼入检索串。这样当用户问“需求分析怎么做”时，也能召回“需求获取”“需求验证”等相邻知识点的资料。

2.2 本课题的数据表单设计

数据库设计遵循“需求分析 → 概念结构设计 → 逻辑结构设计 → 物理结构设计”的三阶段方法。概念结构设计阶段采用自底向上的策略：先对需求文字逐句分析抽取实体，再确定实体之间的联系及其类型，最后集成为全局的 E-R 图。系统的核心 E-R 图如图 4 所示。

图 4 知识管理系统 E-R 图（核心部分）

图 4 知识管理系统 E-R 图（核心部分）

图中涉及的实体与联系说明如表 3 所示。知识条目是核心实体，它通过“属于”联系与分类关联，通过“标注”联系与本体概念关联，通过“包含”联系与附件关联，通过“留存”联系与历史版本关联；标签与知识条目之间是标准的 M：N 联系。

实体 / 联系	类型	关键属性	说明
知识条目	实体	条目 id、标题、摘要、正文、纯文本、资产类型、状态、版本号、累计修改次数	系统的核心实体，正文以 Markdown 存储
分类	实体	分类 id、名称、编码、父分类 id、层级、物化路径	树形结构，用物化路径支持子树查询
标签	实体	标签 id、名称、颜色、使用次数	扁平的交叉分类维度
本体概念	实体	概念 id、编码、名称、别名、父概念 id、物化路径、知识域、难度	本体层次树的节点
本体关系	实体	关系 id、源概念 id、目标概念 id、关系类型、权重	三元组形式的语义关系
附件	实体	附件 id、原始文件名、存储路径、媒体类型、抽取正文	文档类附件上传时同步抽取正文
历史版本	实体	版本 id、版本号、正文快照、变更说明、变更来源、字符增删量	全量快照，支持差异比对与回滚
属于	N : 1 联系	——	1 端（分类）主码放入 N 端（知识条目）作外码
标注	M : N 联系	标注来源、置信度	单独建表；联系自身带有

			属性
关联	M : N 联系	——	知识条目与本体概念的多对多关联

表 3 核心实体与联系说明

转换过程中有两个容易出错的点值得记录。其一是 M : N 联系必须单独转换为一个关系模式，并且联系自身的属性必须放在关联表里：「条目-概念」关联表除了条目 id 与概念 id 之外，还有“标注来源”（人工/自动）与“置信度”两个属性，它们描述的是“某条目标注了某概念”这件联系，而不是条目或概念自身的属性。其二是同一实体集内的联系：「本体概念」自身的上下位关系是自反的 1 : N 联系，用邻接表（父概念 id）即可表达，但查询“某概念的全部后代”时需要递归。因此系统额外维护了物化路径字段 path（形如 /1/5/12/），用一次前缀匹配就能查出整棵子树，这是典型的用空间换查询效率的取舍。

逻辑结构设计阶段对上述模式逐个做了规范化检查。「条目标签」关系模式的码是（条目 id，标签 id），两个属性都是主属性，不存在非主属性对码的部分依赖或传递依赖，满足第三范式与 BCNF；「知识条目」的码是条目 id，各非主属性均直接依赖于码，满足第三范式。这里需要注意的是，如果当初把“分类名称”冗余进知识条目表，就会存在“条目 id → 分类 id → 分类名称”的传递依赖，只能满足第二范式，并会带来分类改名时的大面积更新异常 —— 这正是规范化要避免的问题。完整的 20 张表清单与逐表规范化分析见附件《数据库设计说明书》。

2.3 本课题的体系结构设计

本课题采用分层架构（Layered Architecture）风格。选择这一风格的直接原因是本系统的功能天然具备层次特征：界面呈现、业务处理与数据持久化三者的变化频率与关注点完全不同，分层可以把“界面的调整”与“业务规则的调整”隔离开。另一个原因是分层架构便于团队并行开发 —— 在接口约定之后，前端与后端可以各自推进而互不阻塞。系统的体系结构如图 5 所示。

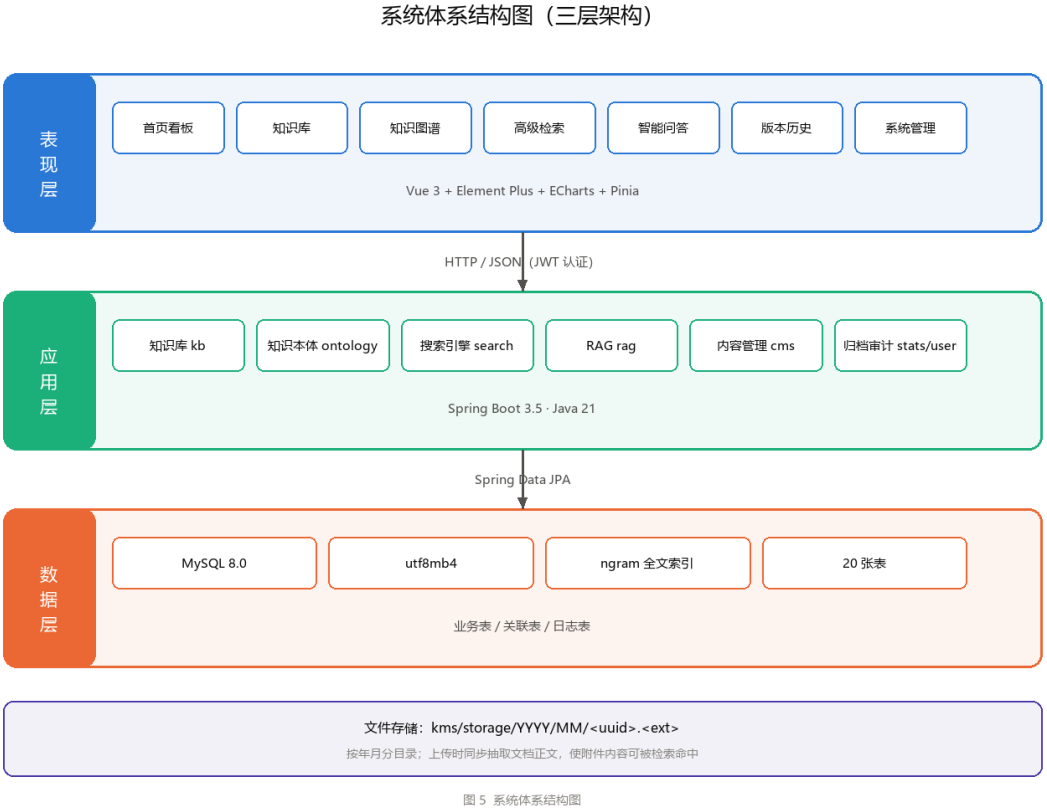


图 5 系统体系结构图

表现层使用 Vue 3 配合 Element Plus 组件库与 ECharts 图表库，通过 Pinia 管理登录状态，通过 Vue Router 的 hash 模式实现前端路由。选择 hash 模式是为了让前端产物可以部署在任意静态服务器上而无需配置服务端重写规则，这在课程设计的演示环境中可以省去很多麻烦。

应用层是本系统的核心，共划分为八个业务模块，各模块的职责与内部结构如图 6 所示。



图 6 后端模块结构图

模块划分遵循高内聚、低耦合的原则，其中有三处划分是经过反复调整后确定的。第一，把“知识本体”从“知识库”中独立出来，因为本体是与具体资料无关的知识结构描述，而知识库是具体的内容管理，两者的变更原因完全不同——本体由领域专家维护，知识库由内容编辑维护。第二，搜索引擎模块复用知识库的分页与装配逻辑而不是另起一套，避免两处维护同一套筛选规则而造成行为不一致。第三，RAG 模块通过 Spring 的事件机制监听知识库的内容变更事件，而不是在知识库的保存逻辑里直接调用 RAG 的索引重建，这样知识库模块不需要知道 RAG 的存在，依赖方向保持单向，两个模块可以独立测试与替换。

数据层使用 MySQL 8.0，字符集为 utf8mb4。中文全文检索使用 MySQL 内置的 ngram 解析器而非独立的搜索引擎，这一决策的取舍将在 3.2 节讨论。文件存储不放在数据库中，而是按“年月/随机文件名”的层级存放在文件系统上，数据库只保存相对路径，避免大字段拖慢主表的查询。

2.4 本课题的主要功能界面设计

界面设计的目标是“每一项功能都能在三步内到达”。为此应用外壳采用左侧分组导航、顶部全局搜索、右侧用户区的固定布局，导航按“知识中心 / 内容

管理 / 系统管理”三组组织，并且按当前用户的角色动态显示 —— 浏览者看不到“内容管理”与“系统管理”两组，减少无关信息对使用者的干扰。系统的主要界面如表 4 所示。

序号	界面名称	主要功能
1	登录页	账号登录与注册；演示账号点击自动填入；展示知识库的真实规模
2	首页看板	统计磁贴、内容趋势、分布图表、本体覆盖率、热门知识与最新动态
3	知识库	分类树、资源类型、知识点、标签四维筛选；卡片式浏览；新建入口
4	知识详情	Markdown 正文渲染、附件预览与下载、关联知识点、相关阅读、版本提示
5	知识图谱	力导向图谱、知识域配色、关系类型筛选、节点详情、学习路径推理
6	知识点管理	本体概念的增删改查、关系维护、内容缺口与覆盖率统计
7	高级检索	全文检索与组合条件筛选、命中高亮、相关度得分、检索建议
8	智能问答	多会话问答、引用溯源、检索过程展开、本体扩展可视化
9	版本历史	版本列表、策略判定说明、版本差异比对、回滚
10	系统管理	用户管理、版本策略配置、检索索引状态、审计日志、检索分析

表 4 系统主要界面一览

首页看板是使用者进入系统后的第一屏，需要在有限的空间内传达“知识库现在有多少内容、内容如何分布、本体覆盖是否完整”三类信息。设计上采用“关键数值用大字号直接呈现、需要比较的数据才画图”的原则 —— 一个孤立的数字做成饼图并不比直接写出来更易读。界面如图 7 所示。

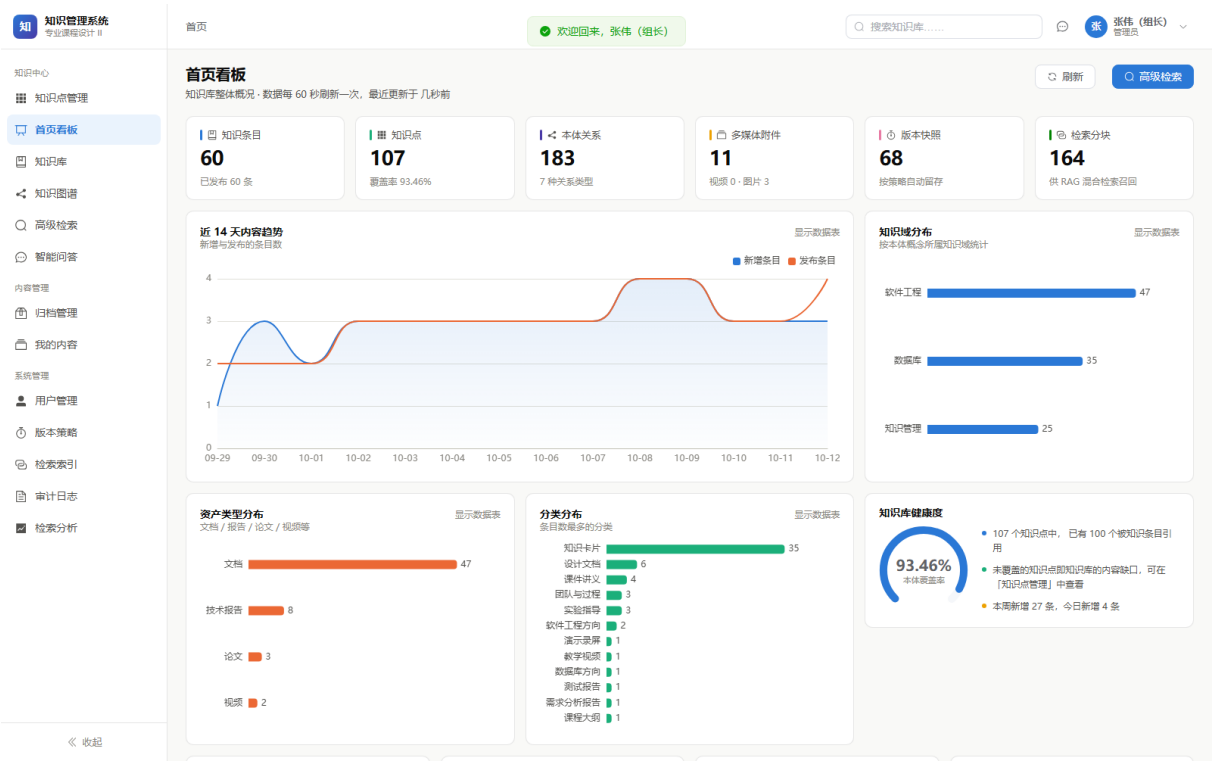


图 7 首页看板界面

知识库界面采用左侧筛选栏加右侧卡片网格的布局，筛选栏提供分类树、资源类型、知识点与标签四个维度，并且每个维度都带有数量提示，使用者可以在筛选之前就知道哪一类内容更多。界面如图 8 所示。

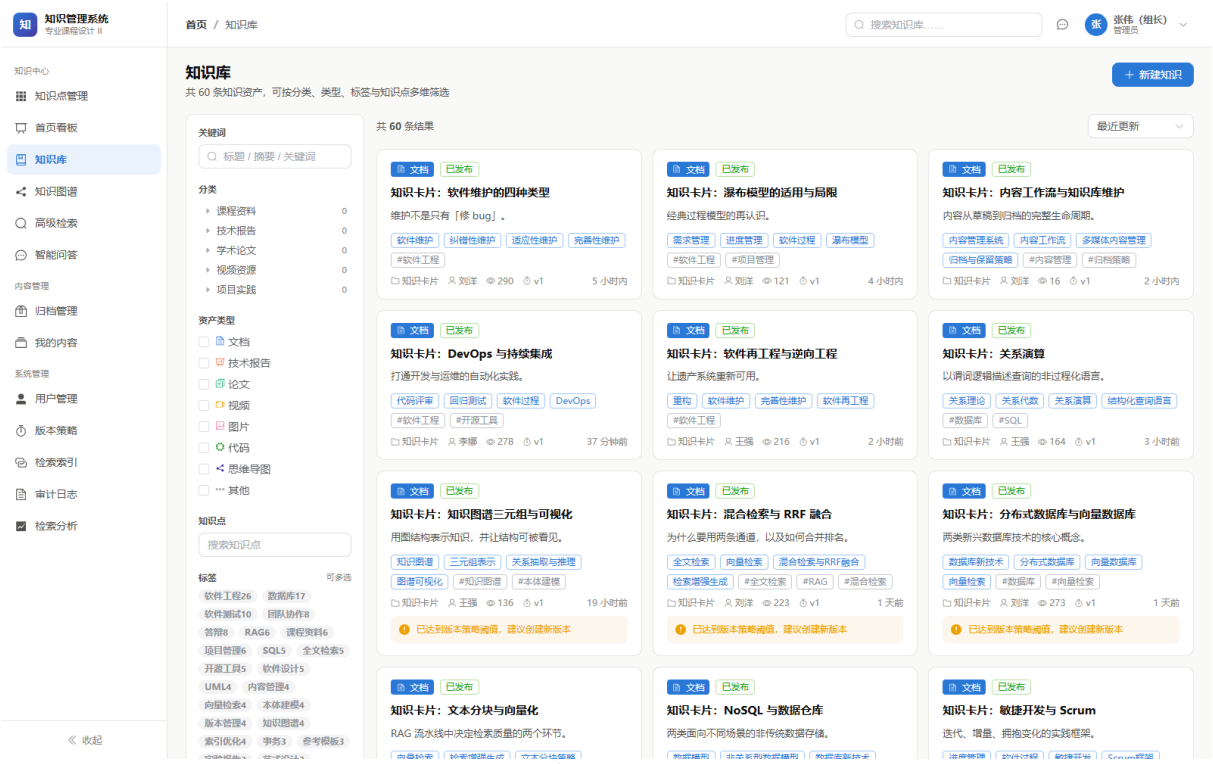


图 8 知识库浏览界面

知识图谱是体现本课题“用本体建模知识的逻辑结构”这一要求的核心界面。设计时考虑的重点不是视觉上的美观，而是如何降低认知负荷：节点颜色编码知识域，使三个域的边界一目了然；节点大小编码该知识点关联的知识条目数量（经过对数压缩，避免个别节点过大）；连线颜色编码关系类型，并且每条边上都标注了关系名称，使得关系的识别不依赖颜色这一个通道，对色觉障碍用户同样可用。界面如图 9 所示。

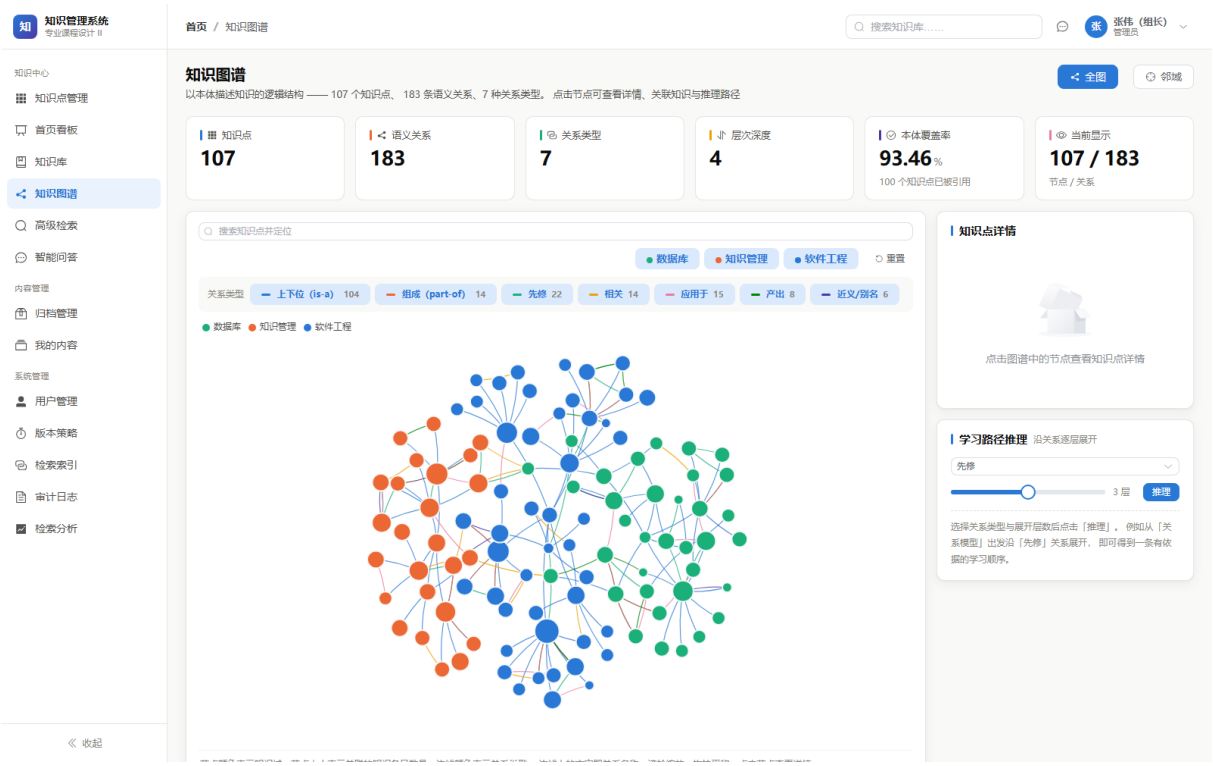


图 9 知识图谱界面

智能问答界面在回答下方给出引用来源列表，每条引用都显示了相关度得分以及它在两条检索通道中的排名，使用者可以据此判断这条引用的可信程度，也可以点击直接跳转到原文核对。界面如图 10 所示。



图 10 智能问答界面

三、相关功能模块详细设计

1 模块内的数据结构设计

本系统涉及的数据结构可以分为三类：用于组织知识的层次结构、用于加速检索的索引结构、以及用于承载算法中间结果的临时结构。以下分别说明其设计与取舍依据。

（1）知识层次结构：邻接表 + 物化路径。本体概念与知识分类都是树，系统同时保存两种表示：邻接表（parent_id）用于表达父子关系，物化路径（path，形如 /1/5/12/）用于一次查询取出整棵子树。只用邻接表时，查询“某个分类及其全部子分类下的知识条目”需要递归或多次查询；引入物化路径后，一次 LIKE 前缀匹配即可得到全部后代节点。代价是节点移动（改变父节点）时需要递归更新所有后代的路径，因此系统把移动操作实现为“先校验是否形成环，再递归刷新后代”，并明确禁止把节点移动到自己的后代之下。

（2）检索候选结构 Candidate。混合检索需要同时记录一条候选分块在两条通道中的表现，才能完成融合并对使用者解释“这条结果为什么被召回”。为此系统定义了一个候选结构，同时保存分块实体、各通道的排名与得分、以及

融合后的最终得分与排名。把各通道的中间结果保留下来而不是只留最终得分，是界面能够展示检索过程明细的前提。

（3）倒排索引：由数据库维护。全文检索的底层结构是倒排索引（词项 → 文档列表），本系统并未自行实现，而是使用 MySQL 的全文索引。需要理解的一点是：中文没有天然的词边界，MySQL 通过 ngram 解析器把连续文本按固定长度（默认 2）切分为词项，因此“软件测试”会被切为“软件”“件测”“测试”三个词项。这带来的直接后果是：查询“软件测试”或“测试”都能命中，但单字查询（如只搜“测”）不会命中，因为单字不构成词项。这一限制已在界面中给出提示。

（4）版本快照结构。版本管理需要回答“某个时间点的内容是什么样”。系统为每个版本保存该时刻的完整正文，而不是保存相对于上一版本的增量补丁。选择全量快照的理由是：知识条目的正文在数十 KB 量级，即使保留 50 个版本也不过几 MB，用存储换取回滚逻辑的简单与可靠是划算的；而增量方案在链条中间断裂时会导致数据无法恢复。每条快照另外保存了正文的 MD5 指纹，用于判断内容是否真的发生了变化 —— 只有格式变动（例如多余空格）时不会产生无意义的版本。

（5）差异比对结构：动态规划矩阵。行级差异比对采用最长公共子序列（LCS）算法，需要一张 (n+1) × (m+1) 的整数矩阵保存子问题的解，其中 n、m 分别是两个版本的正文行数。算法从矩阵右下角回溯生成操作序列，得到每一行是“新增”“删除”还是“未改动”。为避免超长文档造成内存与耗时问题，系统对超过 3000 行的正文退化为“整体替换”标记，并且只向界面返回改动前后各三行的上下文，中间未改动的部分折叠为占位行，使传输量与渲染量都下降一个数量级。

（6）文本分块结构。RAG 检索的最小单位是“块”而不是整篇文档。系统把条目的正文与附件抽取文本按目标长度 420 字符切分，相邻块之间保留 80 字符的重叠。切分的优先级是段落边界优先于句子边界、句子边界优先于长度硬切，目的是让每个块都是一个相对完整的论述单元，而不是从句子中间截断的碎片。每个块记录块序号、来源类型（正文或附件）、字符数以及它对应的向量与向量模型标识。

（7）向量与索引的存储结构。向量以 float32 小端序列的形式存入数据库的 MEDIUMBLOB 字段，相比存成 JSON 文本体积小约 4 倍，且读写不需要解析。检索时把候选向量读入内存，与查询向量计算余弦相似度。每个块同时记录向量维度与产生它的模型标识，检索时只与维度一致的向量比较 —— 因为不同模型产出的向量不在同一个向量空间，跨模型比较的结果没有意义。这一字段也是“切换模型后必须重建索引”这一约束得以自动检查的基础。

数据结构	实现方式	主要用途	取舍理由
知识层次树	邻接表 + 物化路径	分类导航、本体子树查询	用一次前缀匹配换取递归查询，代价是移动节点需刷新后代路径
倒排索引	MySQL ngram 全文索引	中文全文检索召回	免去外部组件部署，索引与业务数据同事务；代价是单字查询不命中
检索候选结构	内存对象（含各通道排名）	混合检索融合与过程解释	保留中间结果使界面能展示“为什么召回这条”
版本快照	完整正文 + MD5 指纹	历史回溯、差异比对、回滚	用存储空间换取回滚逻辑的简单与可靠
差异矩阵	(n+1)×(m+1) 整数矩阵	版本行级差异比对	超长文档退化处理并折叠上下文，控制内存与渲染开销
文本分块	段落优先 + 重叠窗口	RAG 检索的最小召回单位	保证每块是完整论述单元；重叠避免信息落在切分点上
向量存储	float32 小端序列（BLOB）	语义检索的相似度计算	比 JSON 文本省约 4 倍空间，读写无需解析

表 5 主要数据结构及其取舍

2 模块程序流程图设计

本系统的关键算法集中在三处：版本管理的建版判定、检索的混合召回与融合、以及 RAG 的文本分块。以下分别给出流程图并说明各算法之间的结构关系。

（1）版本建版判定流程。版本管理的核心问题是“什么时候该为文档建立一个新的版本快照”。记录太频繁会让版本数量爆炸，记录太少又会失去回溯能力。系统给出的解法是“天数与修改次数双阈值，满足任一即建版”，流程如图 11 所示。

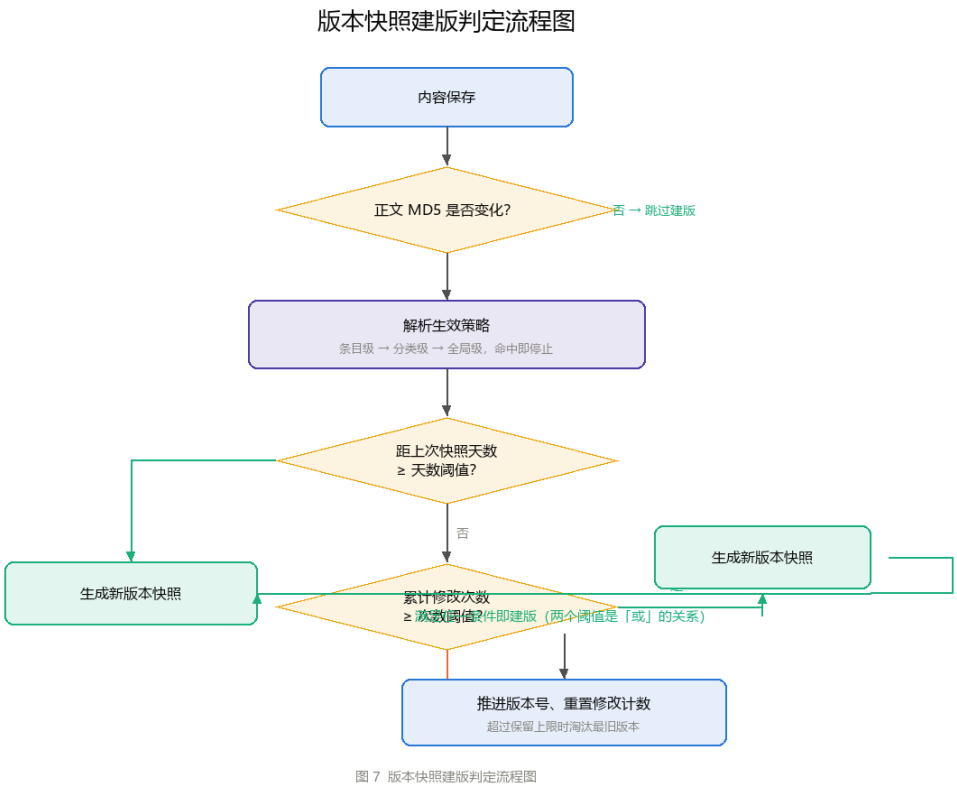


图 11 版本快照建版判定流程图

这一流程中有三个细节值得说明。第一，进入判定之前先用正文的 MD5 指纹判断内容是否真的变化，内容未变则直接跳过，避免产生只有格式差异的空版本。第二，策略的解析顺序是“条目级 → 分类级 → 全局级，命中即停止”，这样既可以为个别高频文档设置更细的粒度，也可以在分类层面统一批量设置，未特殊配置的则落到全局默认值（默认 1 天或 5 次修改）。第三，两个阈值之间是“或”的关系而不是“与”：单用天数阈值时，当天反复修改的文档只会留下一个版本；单用次数阈值时，低频文档可能几个月都不会产生版本。两者取“或”可以在时间跨度与修改密度两个维度上都保证有回溯点。界面在版本历史页把这两个阈值与当前值并排展示，使用者可以直接看出是“时间”还是“改动量”先触发的。

（2）混合检索与 RRF 融合流程。检索模块需要解决“用词不同就搜不到”的问题。系统同时使用两条召回通道并融合其排名，流程如图 12 所示。

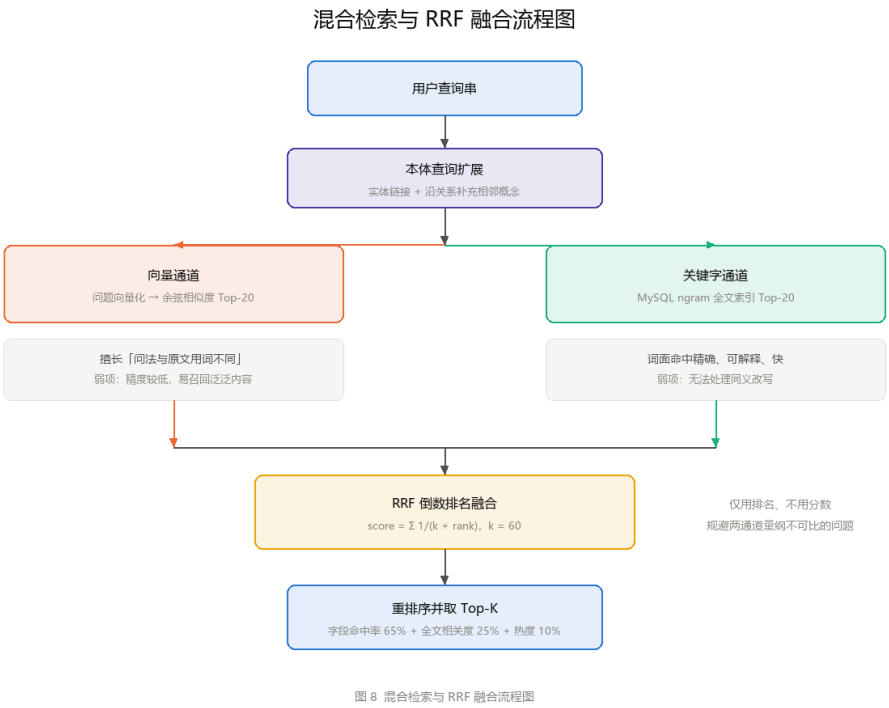


图 12 混合检索与 RRF 融合流程图

两条通道的强弱正好互补：向量通道按语义相似度召回，擅长处理“问法与原文用词不同”的情况，但精度有限，容易召回泛泛而谈的内容；关键字通道按词面命中召回，精确但无法处理同义改写，也容易漏掉表述方式不同的段落。两者的失败模式不重叠，因此组合后的整体效果优于任一单通道。

融合采用倒数排名融合（Reciprocal Rank Fusion，RRF）。之所以不直接相加两侧的分数，是因为余弦相似度落在 0 到 1 之间，而全文索引给出的相关度可能是 0 到 30 的任意值，两者量纲完全不同，加权求和需要大量调参且不稳定。RRF 只用排名、不用分数，按公式对各通道的倒数排名求和，天然规避了量纲问题。它的另一个性质是“奖励两条通道都认可的文档”，而不是“某一条通道特别认可的文档” —— 这正是融合想要的语义。融合之后，结果还会经过一次基于字段命中率的重排序，这一部分在图 12 中未展开，其原理与效果在 3.3 节结合代码说明。

（3）文本分块流程。分块质量直接决定 RAG 的效果，流程如图 13 所示。

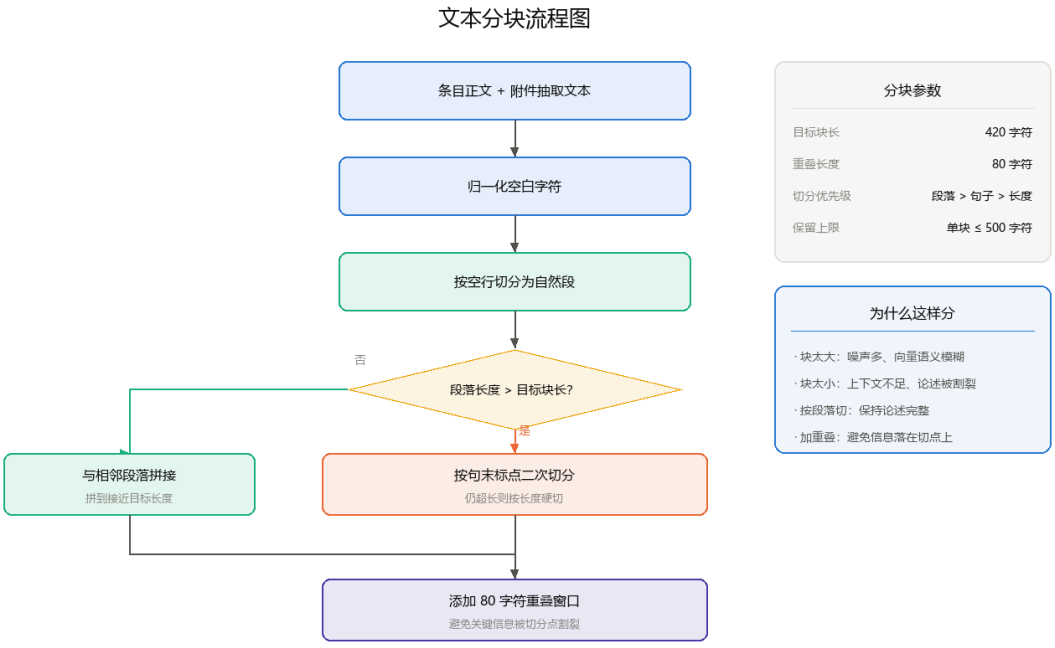


图 9 文本分块流程图

图 13 文本分块流程图

分块的关键在于“在什么位置切”而不是“切多大”。系统的切分优先级是：优先在段落边界收口，段落过长时再按句末标点二次切分，只有连句子都过长时才按长度硬切。这样得到的每一块都是一个语义相对完整的单元。相邻块之间保留 80 字符的重叠，是为了避免关键信息正好落在切分点上被割裂 —— 例如某个定义的完整表述跨越了两块的边界，重叠可以让至少一个块包含完整的定义。

上述三个算法虽然服务于不同的功能，但在结构上是同构的：它们都是“先做一次廉价的判断，把明显不需要处理的情况排除掉，再对确实需要处理的对象执行昂贵操作”。建版判定先用 MD5 排除未变化的正文；混合检索先用扩大召回的方式把候选集取全，再用打分排除弱相关结果；文本分块先按段落切分，只对确实超长的段落才做二次切分。这种“廉价判断前置”的思路贯穿了整个系统的实现。

四、部分核心代码

本节列出系统中五段核心代码。这些代码集中体现了系统在检索质量、版本管理与文本处理上的实现思路，其余代码见附件中的源代码。

4.1 检索结果重排序

这段代码位于 SearchService 中，负责把数据库召回的结果重新打分。需要重排序的原因是：数据库的 ngram 相关度天然偏好长文档 —— 长文档包含的二元组更多，命中数量自然更多，于是标题就叫《软件测试用例设计与覆盖率分析报告》的文档，会被“关于检索技术的长报告”挤到后面去。打分由三部分加权而成，其中字段命中率是主导项。

```
String title = d.getTitle() == null ? "" : d.getTitle().toLowerCase();
String summary = d.getSummary() == null ? "" : d.getSummary().toLowerCase();
String keywords = item.getKeywords() == null ? "" : item.getKeywords().toLowerCase();
String body = item.getPlainText() == null ? "" : item.getPlainText().toLowerCase();

// 标题命中权重最高，摘要与关键词次之，正文只作为补充
double titleCoverage = coverage(terms, title);
double summaryCoverage = coverage(terms, summary + " " + keywords);
double fieldScore = titleCoverage * 0.80 + summaryCoverage * 0.20;

Double raw = scores.get(d.getId());
double fulltextScore = (raw != null && maxScore > 0)
      ? Math.min(raw / maxScore, 1.0) : 0.0;
// 浏览量取对数压缩：避免个别热门文档仅因访问量高就长期占据首位
double popularity = Math.min(1.0, Math.log1p(item.getViewCount()) / Math.log1p(500));

double score = fieldScore * 0.65 + fulltextScore * 0.25 + popularity * 0.10;

// 标题完整包含整个查询串时给一次额外奖励——这类结果几乎总是用户想要的
if (title.contains(terms.get(0).toLowerCase())) {
   score += 0.20;
}
```

代码 1 检索结果的加权打分（SearchService.buildHits 节选）

与之配套的一个关键改动是候选集的放大。最初的做法是让数据库直接按它的相关度返回前 N 条，应用层再对这几条重新排序。但这样做的结果是最相关的那条从一开始就没进入候选集 —— 应用层再怎么排也排不出来。正确的分工是：数据库只负责召回，把候选集放大若干倍取出；排序与分页都在应用层完成。系统采用 6 倍放大（上限 300 条），即为了呈现 10 条结果而读取约 60 条候选。重排序的实际效果对比如图 14 所示。

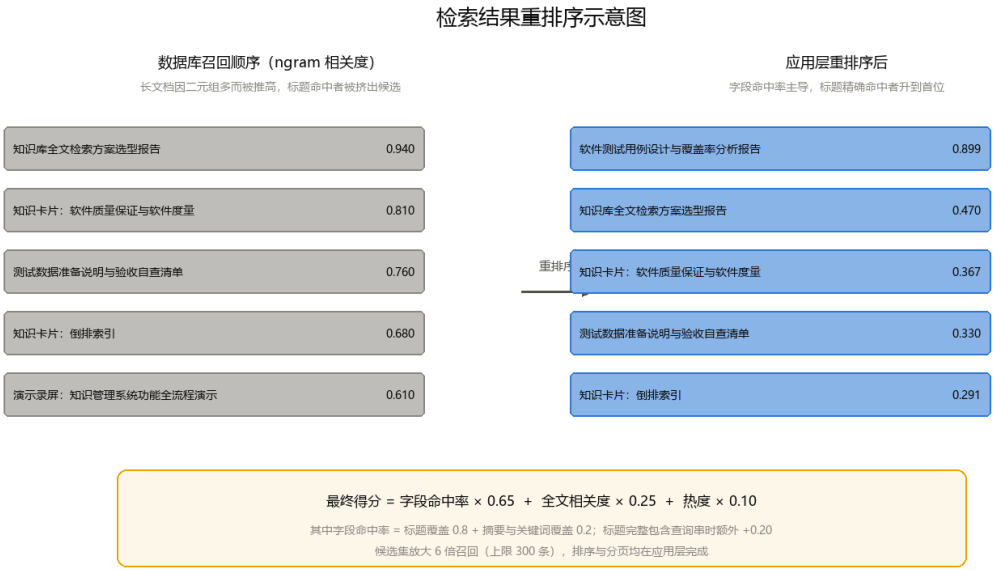


图 11 检索结果重排序示意图

图 14 检索结果重排序效果对比

4.2 混合检索的 RRF 融合

这段代码位于 HybridRetriever 中，实现了倒数排名融合。注意其中的权重处理：关键字通道被乘以 1.15，因为在知识库这类术语密集的语料上，词面命中往往比语义相似更可信。

```
// ---- RRF 融合 ----
List<Candidate> candidates = new ArrayList<>(merged.values());
for (Candidate c : candidates) {
    double score = 0;
    if (c.vectorRank != null) {
        score += 1.0 / (rrfK + c.vectorRank);
    }
    if (c.keywordRank != null) {
        score += 1.0 / (rrfK + c.keywordRank);
    }
    // 关键词通道权重略高：在知识库这类术语密集的语料上，词面命中往往更可信
    if (c.keywordRank != null) {
        score *= 1.15;
    }
    c.fusedScore = score;
}
candidates.sort((a, b) -> Double.compare(b.fusedScore, a.fusedScore));
for (int i = 0; i < candidates.size(); i++) {
    candidates.get(i).fusedRank = i + 1;
}
```

代码 2 RRF 倒数排名融合（HybridRetriever 节选）

公式中的常数 k 取 60。k 的作用是平滑：k 越大，各排名的差距被拉平得越多；k 越小，头部排名的优势越突出。取 60 是信息检索领域的经验值，在本

系统的语料规模下表现稳定。举个直观的例子：文档 A 在向量通道排第 1、关键字通道排第 10，文档 B 在两条通道都排第 3；按 RRF 计算，A 得 1/61 + 1/70 ≈ 0.0307，B 得 1/63 + 1/63 ≈ 0.0317，B 胜出。这正是“两通道都认可”优于“单通道特别认可”的体现。

4.3 版本策略的双阈值判定

这段代码位于 VersionPolicyService 中，实现了“天数或修改次数满足其一即建版”的判定。判定结果不只是返回一个布尔值，而是连同判定依据一起返回，使界面能够向使用者解释“为什么不建版”或“为什么现在建版”。

```
    if (!policy.isEnabled() || !policy.isAutoSnapshotEnabled()) {
        d.setDue(false);
        d.setReason("该条目适用策略「%s」，但自动快照已关闭，仅在手工创建/发布时生成版本"
                .formatted(policy.getPolicyName()));
        return d;
    }

    boolean byDays = days >= policy.getIntervalDays();
    boolean byCount = d.getPendingChangeCount() >= policy.getMinChangeCount();

    if (byDays && byCount) {
        d.setDue(true);
        d.setReason("距上次快照 %d 天（≥ %d 天）且累计修改 %d 次（≥ %d 次），满足建版条件"
                .formatted(days, policy.getIntervalDays(),
                        d.getPendingChangeCount(), policy.getMinChangeCount()));
    } else if (byDays) {
        d.setDue(true);
        d.setReason("距上次快照 %d 天，已达到 %d 天的天数阈值"
                .formatted(days, policy.getIntervalDays()));
    } else if (byCount) {
        d.setDue(true);
        d.setReason("累计修改 %d 次，已达到 %d 次的次数阈值"
                .formatted(d.getPendingChangeCount(), policy.getMinChangeCount()));
    } else {
        d.setDue(false);
        d.setReason("距上次快照 %d 天（阈值 %d 天），累计修改 %d 次（阈值 %d 次），尚未满足建版条件"
                .formatted(days, policy.getIntervalDays(),
                        d.getPendingChangeCount(), policy.getMinChangeCount()));
    }
```

代码 3　版本建版的双阈值判定（VersionPolicyService.evaluate 节选）

把“理由”作为返回值的一部分，而不是在界面层重新拼装描述文字，是为了保证判定逻辑与解释文字永远一致 —— 如果两处各写一遍，一旦阈值语义调整就很容易出现“代码判定了但界面说法不对”的情况。

4.4 版本差异比对

这段代码位于 DiffService 中，用最长公共子序列算法计算两个版本的行级差异。先用动态规划求出 LCS 长度矩阵，再从右下角回溯生成操作序列。相比逐行比较，LCS 能够正确识别“中间插入了若干行”这类改动。

```
int n = oldLines.size();
int m = newLines.size();
int[][] lcs = new int[n + 1][m + 1];

// 自右下向左上填表：lcs[i][j] 表示 oldLines[i..] 与 newLines[j..] 的最长公共子序列长度
for (int i = n - 1; i >= 0; i--) {
   for (int j = m - 1; j >= 0; j--) {
      lcs[i][j] = oldLines.get(i).equals(newLines.get(j))
            ? lcs[i + 1][j + 1] + 1
            : Math.max(lcs[i + 1][j], lcs[i][j + 1]);
   }
}

// 回溯生成操作序列
int i = 0;
int j = 0;
while (i < n && j < m) {
   if (oldLines.get(i).equals(newLines.get(j))) {
      result.add(CmsDTO.DiffLine.of("SAME", i + 1, j + 1, oldLines.get(i)));
      i++;  j++;
   } else if (lcs[i + 1][j] >= lcs[i][j + 1]) {
      result.add(CmsDTO.DiffLine.of("REMOVE", i + 1, null, oldLines.get(i)));
      i++;
   } else {
      result.add(CmsDTO.DiffLine.of("ADD", null, j + 1, newLines.get(j)));
      j++;
   }
}
// 尾部剩余行分别为纯删除与纯新增
while (i < n) { result.add(CmsDTO.DiffLine.of("REMOVE", i + 1, null, oldLines.get(i)));
i++; }
while (j < m) { result.add(CmsDTO.DiffLine.of("ADD", null, j + 1, newLines.get(j))); j++; }

return collapseContext(result);
```

代码 4 基于 LCS 的行级差异比对（DiffService 节选）

算法的复杂度是 O(n·m) 时间与 O(n·m) 空间。对课程设计场景的文档规模（数百行）而言这个开销可以忽略，但仍需对异常输入做保护：当任一版本的正文超过 3000 行时，算法退化为“整体标记为新旧替换”，避免用户对比两份长文档时界面长时间无响应。最后一行调用的 collapseContext 用于折叠未改动的上下文，只保留改动前后各三行，其余用一条占位行表示，使返回给界面的数据量下降一个数量级。

4.5 文本分块与重叠窗口

这段代码位于 ChunkingService 中，负责把长文本切分为适合检索的块。分块策略的核心是“段落优先、句子次之、长度兜底”，并在此基础上添加重叠窗口。

```
// 1) 先按自然段聚合，尽量在段落边界收口
List<String> paragraphs = splitParagraphs(normalized);
List<String> blocks = new ArrayList<>();
StringBuilder current = new StringBuilder();

for (String para : paragraphs) {
   // 单段就超长：先把已有内容收口，再把该段按标点切开
   if (para.length() > size) {
      flush(current, blocks);
      blocks.addAll(splitLongParagraph(para, size, ov));
      continue;
   }
   if (current.length() + para.length() + 1 > size && current.length() > 0) {
      flush(current, blocks);  // 接近目标长度，在段落边界切分
   }
   if (current.length() > 0) {
      current.append('\n');
   }
   current.append(para);
}
flush(current, blocks);

// 2) 加重叠窗口：把上一块的尾部拼到下一块开头，避免关键信息落在切分点上
List<Chunk> chunks = new ArrayList<>();
String previousTail = "";
for (int i = 0; i < blocks.size(); i++) {
   String content = blocks.get(i);
   if (ov > 0 && !previousTail.isEmpty()) {
      content = previousTail + "\n" + content;
   }
   if (content.length() > size + ov + 8) {
      content = content.substring(0, size + ov);  // 拼接后超长则截断，保证单块规模可控
   }
   chunks.add(new Chunk(i, content, content.length()));
   previousTail = ov > 0 && blocks.get(i).length() > ov
         ? blocks.get(i).substring(blocks.get(i).length() - ov)
         : blocks.get(i);
}
```

代码 5 文本分块与重叠窗口（ChunkingService 节选）

重叠窗口的实现方式是“记录上一块的尾部，拼到下一块的开头”。这样做的直接后果是块与块之间存在重复内容，检索时同一段话可能被多个块命中，因此组装上下文时还需要按条目去重（这一处理在 RagService 中完成）。参数上取目标块长 420 字符、重叠 80 字符，是在“块太大导致向量语义模糊”与“块

太小导致上下文不足”之间的折中，重叠比例约为 19%，属于常见的 10% 到 20% 区间。

五、软件测试及其结果分析

5.1 测试环境

项目	配置
操作系统	Windows 11
运行时	JDK 23（编译目标 Java 21）、Node.js 24
构建工具	Maven 3.9、Vite 6
数据库	MySQL 8.0.42（字符集 utf8mb4，ngram_token_size = 2）
浏览器	Chromium（由 Playwright 驱动，用于界面自动化测试）
测试数据	60 条知识条目、107 个知识点、183 条本体关系、11 个附件、164 个检索分块

表 6 测试环境

5.2 测试策略

测试采用自底向上的分层策略：先对工具类（文本处理、文件抽取、差异比对）做单元测试，再对服务类之间的协作做集成测试，最后对完整系统做系统测试与验收测试。考虑到本系统的功能大量依赖数据库与浏览器环境，测试工作的重心放在后两个层次，并且通过两个可重复执行的自动化脚本来承载：接口测试脚本覆盖服务端的全部功能链路，界面测试脚本用真实浏览器验证页面渲染与关键交互。这两个脚本的价值不仅在于一次性的验证，更在于它们可以在任何环境下重新执行 —— 换一台机器部署后，跑一遍脚本即可确认环境是否正常。

测试用例的设计综合使用了黑盒与白盒两类方法。黑盒方面主要采用等价类划分与边界值分析，下面各举一个本系统设计中的真实例子。

等价类划分的例子是“创建知识条目”的标题输入。标题的合法范围是 1 到 255 个字符，据此可以划分出有效等价类与若干无效等价类，原则上每个无效等价类都要单独设计一个用例，因为一次只能包含一个非法输入，否则第一个非法输入被拒绝之后其余的无法验证。划分结果如表 7 所示。

输入项	等价类	类型	代表值	预期结果
标题	长度 1~255 且非空	有效	需求分析实践总结	创建成功
标题	空串	无效	（空）	提示“请输入标题”
标题	长度超过 255	无效	256 个字符的串	提示“标题过长”
标题	长度不足 2	无效	知	提示“标题长度 2~255 个

				字符”
分类	系统中存在的分类 id	有效	技术报告	正常关联
分类	不存在的分类 id	无效	99999	分类字段置空，不报错
标签	长度 1~64	有效	软件测试	自动创建并关联
标签	长度超过 64	无效	65 个字符的串	该标签被静默忽略

表 7 标题与关联字段的等价类划分

边界值分析的例子是版本策略的天数阈值判定。判定式为“距上次快照的天数 ≥ 天数阈值”，默认阈值为 1 天。按照边界值分析方法，取略小于边界、恰好等于边界、略大于边界三个值，即 0 天、1 天、2 天。测试结果证实：0 天不建版，1 天与 2 天均建版。这个用例之所以重要，是因为“≥”与“>”的一字之差在代码评审中极易被忽略，而它恰好是版本管理功能最容易出错的地方。另一个边界用例是知识图谱的层次深度，测试了 1 层、3 层与系统上限 5 层的推理展开，均返回正确结果。

5.3 功能测试与结果

接口测试通过脚本 smoke_test.py 执行，共 78 项断言，覆盖八个功能模块，全部通过。测试结果汇总如表 8 所示。

测试模块	主要断言内容	结果
认证与权限	管理员登录返回 JWT、错误口令被拒绝、浏览者访问管理接口返回 403、匿名可读公开接口	通过
知识本体	知识点数 ≥ 20、关系类型 ≥ 5、层次树可获取、图谱节点与边非空、实体链接能识别知识点、先修关系推理返回正确路径	通过
搜索引擎	全文检索命中并返回相关度、命中词高亮、术语检索（需求分析/范式/RAG/索引优化）均命中、无结果时给出建议、按类型过滤生效	通过
知识库	分页列表、分类与作者装配、本体概念标注、标签装配、详情返回正文、分类树与热门标签	通过
版本管理	版本列表、策略判定给出解释、策略列表	通过
内容管理	创建条目并自动生成 v1 基线版本、自动抽取关键词、自动标注本体概念、连续修改后计数增长、手工建版、版本差异比对、回滚生成新版本、发布、归档、恢复、删除	通过
RAG 问答	提问返回回答、引用来源非空、引用含片段与得分、引用标注通道排名、检索元数据完整、本体扩展生效、双通道均有候选、无资料时如实说明、会话列表、索引状态	通过
统计与审计	归档列表、统计总览、分布数据、发布趋势、审计日志、系统能力自检	通过

表 8 接口测试结果汇总

界面测试通过脚本 ui_check.mjs 执行，用真实浏览器打开 15 个页面，共 38 项断言，全部通过。测试不仅检查页面元素是否存在，还收集浏览器的运行期错误与失败的网络请求，这可以捕捉到“构建通过但运行时报错”这类仅在浏览器中才会暴露的问题。测试同时自动生成各页面截图，因此截图存在本身即意味着该页面通过了检查。部分测试结果如表 9 所示。

测试项	验证内容	结果
登录与鉴权	登录页渲染、演示账号展示、登录后跳转首页、无运行期错误	通过
首页看板	6 个统计磁贴、4 个 ECharts 画布已绘制、本体覆盖率数值显示正确	通过
知识库	条目卡片列表、分类树渲染、无运行期错误	通过
知识图谱	画布绘制、7 种关系类型图例、显示节点数 107 / 关系数 183、点击节点加载详情且显示名称、无运行期错误	通过
高级检索	返回结果、命中词高亮、展示检索通道与耗时、标题命中者排在最前、无运行期错误	通过
智能问答	提问后返回回答、回答附带引用、离线模式有明确降级提示、检索过程面板可展开	通过
知识详情与版本	进入详情页、Markdown 正文渲染、标签与知识点展示、版本页策略判定说明、版本列表有数据	通过
管理端	索引状态、版本策略、用户管理、检索分析、审计日志五个页面均正常，无运行期错误	通过

表 9 界面测试结果汇总

界面的实际效果可参考图 7 至图 10 以及附件中的其余截图。图 15 是版本历史界面，它把“天数阈值”与“次数阈值”的当前值与阈值并排展示，使用者可以直接看出是哪一个条件触发了建版，这是版本管理功能在界面上的直观体现。

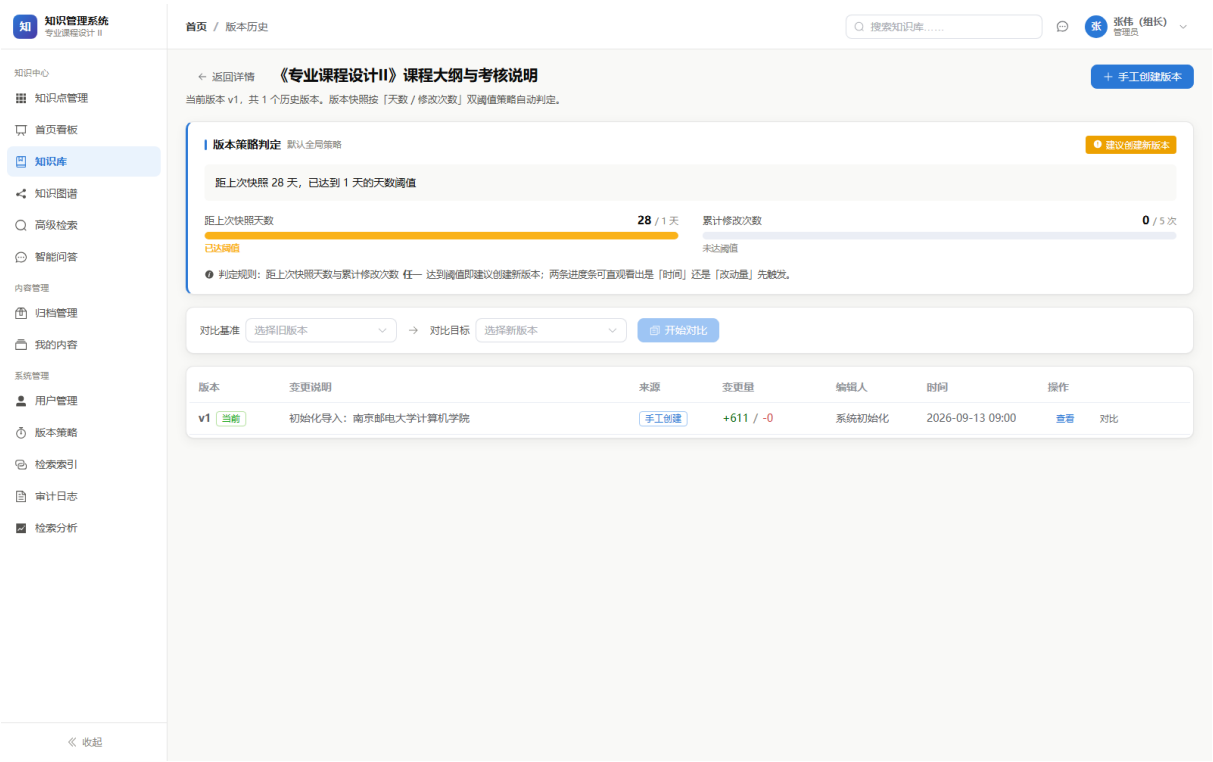


图 15 版本历史界面（含策略判定说明）

图 16 是高级检索界面。搜索结果中命中的词以背景色高亮，每条结果下方给出正文摘要片段，鼠标悬停在相关度得分上会显示得分的构成明细，便于使用者理解排序依据。

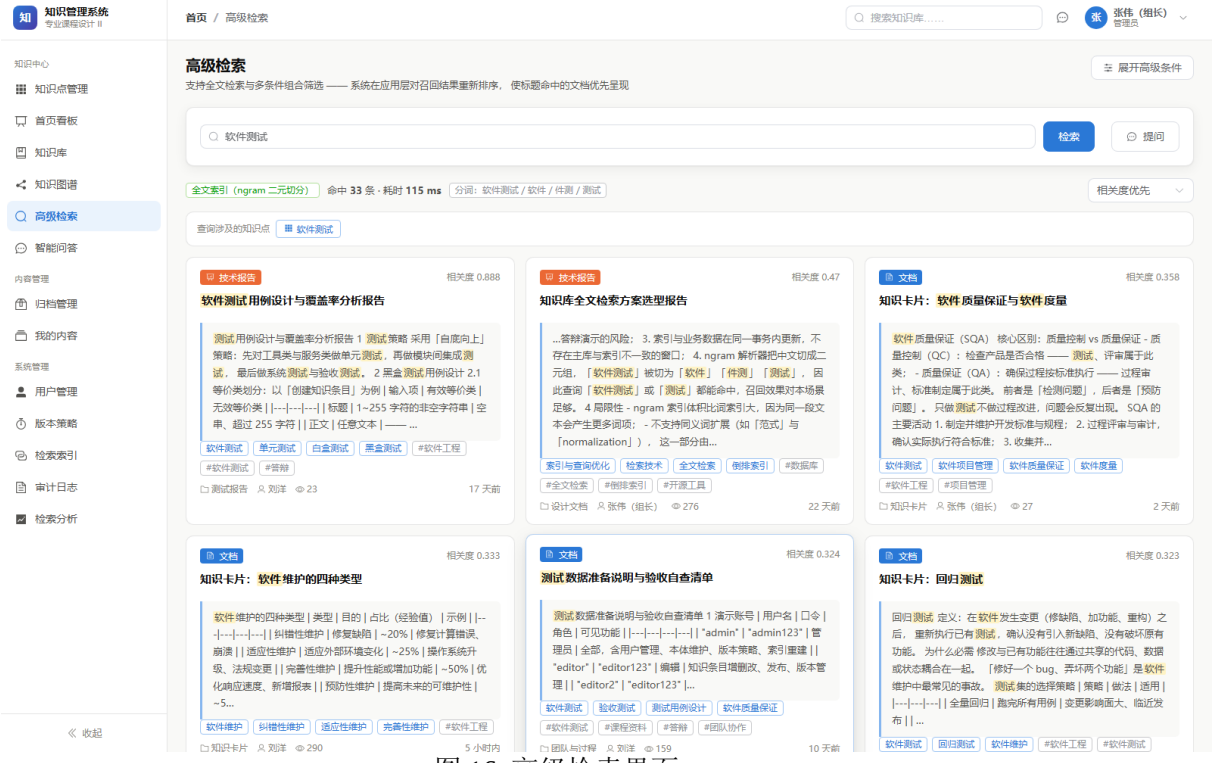


图 16 高级检索界面

5.4 性能测试与结果分析

在 60 条知识条目、164 个检索分块的测试数据规模下，关键操作的实测耗时如表 10 所示。

操作	实测耗时	说明
全文检索（含重排序）	约 20 ms	数据库召回约 60 条候选，应用层打分后本地分页
向量索引全量构建	约 1.2 s	60 条知识切分为 164 个分块并逐条向量化
RAG 问答（检索阶段）	约 80 ms	含本体扩展、双通道召回与 RRF 融合
RAG 问答（生成阶段）	约 7 ms	离线抽取式回答；在线大模型受网络影响
知识图谱渲染	约 6 s	107 个节点、183 条边的力导向布局

表 10 关键操作实测耗时

对上表数据可以做几点分析。检索的耗时远低于任务书隐含的“快速定位”要求，说明在当前数据规模下使用 MySQL 全文索引而非独立搜索引擎是合理的 —— 作为对比，用 LIKE 模糊匹配做全表扫描的耗时在数百毫秒量级，且无法按相关度排序。向量索引构建耗时约 1.2 秒，这是一次性成本且已通过异步执行与用户的保存操作解耦，但若数据规模增长一个数量级，这个耗时将变得不可忽略，届时需要改为增量索引或引入专用的向量索引结构。

知识图谱的渲染耗时明显高于其他操作，这是力导向布局的计算成本，与数据量呈超线性关系。在本系统的规模下 6 秒是可以接受的，但这也解释了为什么系统在节点数超过 80 时关闭了布局动画 —— 动画期间的持续重绘会让交互明显卡顿。这一取舍是实测后确定的，而非预先设计。

六、课题完成过程中遇到的问题及解决方法

本节记录开发过程中遇到并解决的七个典型问题。按性质分，其中三个属于框架层面的陷阱（问题 1、3、4），两个属于算法设计层面的判断失误（问题 2、5），两个属于工程一致性层面的隐患（问题 6、7）。这七条的共同点是：它们都不会让程序崩溃，而是让结果静默地不正确 —— 这类问题恰恰最难发现，也最有记录价值。

问题 1：向量索引重建整体失败，提示“Executing an update/delete query”

现象：系统首次启动时自动构建向量索引，日志中对每一条知识都报出“Executing an update/delete query”异常，导致 60 条知识的索引全部构建失败。

定位：异常指向 RagIndexService 中删除旧分块的语句。该语句带有 @Modifying 注解，必须在事务中执行，而索引重建方法本身已经标注了 @Transactional。进一步检查发现，问题出在调用方式上 —— 全量重建方法是通过 this.reindexItem(...) 的形式调用单条重建方法的，这是类内部的自调用，不经过 Spring 的代理对象，因此 @Transactional 根本没有生效，方法运行时确实没有事务。

解决：通过 ObjectProvider 注入自身的代理引用，把内部调用改为 selfProvider.getObject().reindexItem(...)，使调用真正经过代理，事务边界得以建立。同时把每个条目的重建放在独立事务中，这样单条失败不会影响整批。

验证：重启后日志显示“向量索引构建完成：60 条知识 → 164 个分块，耗时 1233 ms”，数据库中 rag_chunk 表的 164 条记录全部带有向量数据。

问题 2：检索结果排序异常，“最相关的一条根本排不出来”

现象：检索“软件测试”时，标题精确包含该词的《软件测试用例设计与覆盖率分析报告》排在第 5 位，而前三名是内容中偶尔提到该词的文档。

定位：最初的实现是让数据库按 ngram 相关度返回前 N 条，应用层再对这几条重排序。调试后发现，ngram 相关度偏好长文档 —— 长文档包含的二元组多，命中自然多，于是那篇标题命中的文档从一开始就没有进入候选集。应用层的重排序逻辑本身没有错，错的是它作用的范围：再怎么排也排不出一个不在候选集里的文档。

解决：重新划分职责 —— 数据库只负责召回，把候选集放大 6 倍取出（上限 300 条），排序与分页全部在应用层完成。同时把打分函数从“数据库得分加标题加权”改为“字段命中率 × 0.65 + 全文相关度 × 0.25 + 热度 × 0.10”，让标题命中成为主导因素。

验证：修改后检索“软件测试”的首位结果为《软件测试用例设计与覆盖率分析报告》，得分 0.899，远超第二位的 0.470。这一对比已作为图 14 收入报告。

问题 3：知识图谱页面一片空白

现象：知识图谱页面显示了统计数值“107 个知识点、183 条关系”，说明数据已经取到，但画布上什么都没有。控制台没有任何报错。

定位：在浏览器中读取画布的实际尺寸，发现其高度为 0。组件的设计是接受一个高度属性并以内联样式设置容器高度，在图表卡片中使用时传入的是数值（如 :height="286"），会拼接成 "286px"，一切正常；而在图谱页面中写的是 height="560"，传入的是字符串 "560"，拼接逻辑直接原样输出。而 CSS 中 "560" 缺少单位，是一条非法声明，会被浏览器直接丢弃，于是容器高度为 0，图表便以一个零高度的画布完成了初始化。

解决：在组件内部对高度做归一化，凡是纯数字形式的输入一律补上 px 单位，同时保留对 "100%"、"auto" 这类合法 CSS 值的支持。

验证：修正后画布高度为 560 像素，图谱正常渲染；进一步调整力导向布局的斥力、边长与初始缩放比例后，107 个节点完整落在可视区内。这个问题的教训是：动态设置尺寸时，数值与带单位的字符串必须区别对待，而浏览器对非法 CSS 是静默忽略的，不会给出任何提示。

问题 4：检索结果的高亮标签出现嵌套

现象：搜索结果中偶尔出现 <mark><mark>软件</mark>测试</mark> 这样的嵌套标记，导致部分文字的高亮背景色深浅不一。

定位：高亮的最初实现是逐个查询词依次做替换，先用长词替换（避免"需求"抢先匹配掉"需求分析"），再用短词替换。问题在于第二次替换是在**已经插入了标记的字符串**上进行的，于是"软件"匹配到了 "<mark>软件测试</mark>" 内部的那段文字。

解决：把逐个替换改为单趟扫描 —— 先在原文上找出所有查询词的出现区间，合并重叠与相邻的区间，再一次性拼接结果。这样每个字符最多被标记一次，从根本上消除了嵌套。

验证：检索"软件测试""需求分析"等词，界面高亮正常，自动化界面测试断言页面中存在高亮标记且无异常。这个问题的教训是：对文本做多次插入式变换时，被插入的内容会污染后续的匹配，应当先确定所有变换位置再统一执行。

问题 5：统计图表中"发布量"曲线与"新增量"曲线对不上

现象：首页看板的趋势图中，某一天的"新增条目"为 0，但"发布条目"却是 5，两条曲线在同一时间轴上的含义出现矛盾。

定位：检查统计代码发现，“新增”与“发布”两个序列的过滤条件不一致 —— “新增”过滤了逻辑删除的条目，而“发布”没有。系统中删除条目采用逻辑删除（置删除标志位），因此测试过程中创建后又删除的条目仍然会出现在“发布”曲线上。

解决：统一两个序列的过滤口径，都排除逻辑删除的条目。

验证：修正后两条曲线的关系符合预期，并且由于系统中约三分之一的条目是“先起草、后发布”，两条曲线各自具有独立的信息量而不是完全重合。这个问题的教训是：同一张图上的多个序列必须使用一致的数据口径，否则图形会传达错误的结论 —— 而图形上的错误比文字上的错误更难被发现。

问题 6：分类的条目计数不随条目增删更新

现象：在某个分类下新建知识条目后，分类树上该分类显示的条目数不变；删除条目后同样不减少，只有手动触发全量重算才会更新。

定位：检查代码发现，用于增量调整计数的 `CategoryRepository.addItemCount` 方法**已定义但从未被调用** —— 属于“方法存在被误当成逻辑已实现”这类问题。条目新建、删除、改分类三条路径上都没有同步这个冗余字段，因此 `kb_category.item_count` 长期停留在数据导入时的快照值。

解决：新增 `adjustCategoryCount(categoryId, delta)` 方法，在三条路径上分别调用：新建时 +1、逻辑删除时 −1、改分类时原分类 −1 且新分类 +1。其中改分类需要在 `applyFields` 覆盖 `categoryId` 之前先记下原值，否则拿不到旧分类。计数口径与全量重算保持一致 —— 只统计未逻辑删除的条目、与条目状态无关，因此归档、发布这些状态流转不需要调整计数。

验证：编写对照脚本分别验证三条路径，创建后分类计数由 1 变为 2、删除后由 2 恢复为 1、改分类后原分类保持不变而新分类由 6 增至 7，全部正确同步。

这个问题的发现方式值得记录：它不影响任何功能的主流程，接口测试与界面测试都测不到。是在做文档一致性核对时，临时写了一段“对比创建条目前后分类树返回的条目数”的脚本才暴露出来的。这提示了冗余字段的通用风险

—— **它不会报错，只会静默地不准**。凡是引入冗余字段，就必须同时想清楚：谁负责维护它、在哪些路径上维护、以及万一漏了怎么修复。

问题 7：换一种方式启动后，之前上传的附件“不见了”

现象：从项目根目录启动服务时，上传的附件会写到项目之外的一个目录；而在子目录中启动时写到预期位置。系统不报任何错误，但先前上传文件的下载链接会失效。

定位：文件存储根目录在配置中是一个相对路径，相对路径的基准是**进程的工作目录**，而工作目录取决于从哪里启动服务。两种启动方式下工作目录不同，解析出的绝对路径自然不同。

解决：一是让启动脚本在启动前显式切换到固定的工作目录，使附件始终落在项目内部的存储目录；二是在服务启动时把解析后的绝对路径打印到日志，使这类问题在日志里一眼可见，而不必靠推测。

验证：两种启动方式下日志显示的存储根目录一致，上传与下载功能均正常。这个问题的教训与问题 1 有共通之处：凡是“隐式依赖运行环境”的配置，都应当把实际生效的值显式暴露出来。

七、总结

本次课程设计以“知识管理系统”为题，完成了一套包含知识库、知识本体、搜索引擎、检索增强问答与内容管理五大模块的完整系统。系统实现了任务书要求的全部功能，在本体规模上超出要求五倍以上，并通过两套可重复执行的自动化测试脚本验证了功能正确性。回顾整个开发过程，以下几点体会最为深刻。

7.1 关于查阅资料与自主学习

本次设计中遇到的技术点绝大多数不在教材范围内：MySQL 的 ngram 全文索引、倒数排名融合算法、检索增强生成的完整流水线、最长公共子序列在版本比对中的应用、Java2D 与 PDFBox 生成文档等。这些内容都是通过查阅官方文档、技术博客与开源项目源码学到的。体会最深的一点是：**官方文档永远优先于二手教程**。例如在排查全文检索为何偏好长文档时，搜索引擎上能找到的解释大多停留在“TF-IDF 就是这样”，而 MySQL 官方文档中对 ngram 解析

器的切分规则与相关度计算方式有明确说明，正是这部分内容让我确认了“二元组数量决定相关度”这一根本原因，从而选择了“数据库只召回、应用层排序”的正确解法。

另一个体会是：遇到问题时先缩小范围，再定位原因。本报告中记录的六个问题，无一例外都是通过“先复现、再二分、后定位”解决的。例如知识图谱空白的问题，如果一上来就怀疑 ECharts 配置写错，很可能在错误的方向上耗费大量时间；而实际做法是先确认数据是否已到达（统计数值显示正常），再确认图表是否初始化（画布元素存在），最后测量画布的尺寸（高度为 0）——三次判断就锁定了原因。

7.2 关于团队交流与协作

本项目由六人小组协作完成，分工覆盖需求分析、数据库设计、后端开发、前端开发与测试文档。协作中最有价值的一项实践是**在写代码之前先把接口约定下来**。我们在第一周就确定了统一的响应结构、状态码约定与全部接口的路径、方法、参数与返回结构，此后前端与后端可以各自推进而互不阻塞。这个约定带来的直接收益是：整个开发过程中几乎没有出现“接口对不上”导致的返工，联调阶段的主要工作是确认细节而非重新设计。

协作中也有教训。在开发中期，前端同学需要按知识点筛选的接口，而后端同学实现的是一个返回全部知识点的接口，双方各自理解正确但没有对齐，直到联调时才发现。事后我们把这个问题的根因归结为“接口文档没有及时更新”，于是在接口文档中增加了参数说明与调用示例，并要求任何接口变更都必须同步更新文档。这件事让我认识到：团队协作中的问题往往不是能力问题，而是信息同步问题。

7.3 关于项目管理与经济决策

本次实践让我第一次在真实项目中体会到“技术选型是一种经济决策”。选择用 MySQL 全文索引而不是部署 Elasticsearch，并不是因为前者在技术上更先进，而是因为在这个项目的约束条件下它的总成本更低：数据规模只有万级，MySQL 完全能够胜任；索引与业务数据同库同事务，省去了数据同步环节与不一致的风险；更重要的是省去了部署与维护一个中间件的成本 —— 对于一次两周的课程设计而言，这部分成本的边际收益是负的。

同样的权衡出现在向量存储上。常用的做法是引入向量数据库，但在万级分块的规模下直接做暴力比对只需要几十毫秒，引入专用组件带来的运维复杂度远大于它节省的查询时间。因此系统选择用 BLOB 字段保存向量、在内存中比对。这个决策有明确的适用边界：如果数据量增长到百万级，暴力比对不再可行，届时引入近似最近邻索引才是划算的。把这些边界条件写进设计文档，比单纯给出结论更有意义 —— 它让后续的维护者知道“什么时候这个决策会失效”。

在进度管理上，我们采用了两周的迭代计划，每日同步进度与阻塞项。识别出的最大风险是“演示环境可能没有网络”，对应的应对措施是为智能问答设计降级路径：在没有配置大模型接口或调用失败时，自动切换为离线抽取式回答，直接抽取检索到的原文要点并按来源编号列出。这个降级不是“聊胜于无”的兜底 —— 它零幻觉、可逐句核对，在断网环境下依然能够完整演示问答功能的全部链路。事后看，正是这个决策保证了答辩演示的确定性。

7.4 关于可持续发展因素的考虑

任务书要求在设计中考量安全与环保等可持续发展因素。在安全方面，系统做了以下几项工作：采用基于角色的访问控制，写操作必须在服务端校验权限，前端的按钮隐藏只作为体验优化而不作为安全手段；口令使用 BCrypt 加盐散列存储，数据库中不保存明文；文件上传按扩展名白名单校验，并对文件路径做规范化检查以防止路径穿越；文本渲染关闭了 HTML 直通，避免多人协作场景下的存储型 XSS —— 这一点尤其重要，因为知识内容由多个编辑者产生，若允许内联 HTML，任何编辑者都能注入脚本影响其他用户。

在环保与资源节约方面，系统的主要设计考量是“避免不必要的计算与存储”。版本快照在生成前先用正文指纹判断内容是否变化，相同则跳过，避免产生只有格式差异的空版本；索引重建通过事件机制异步触发且只在内容真正变化时执行，而不是每次保存都全量重算；检索结果的弱匹配判定避免了把大量不相关结果渲染给用户，既节省了网络传输也节省了客户端的渲染开销；分块策略中的长度上限控制了单块的规模，避免个别超长文本显著增加向量化与存储成本。这些优化单独看都很小，但它们是同一思路的体现：在流程的每一步都先做一次廉价的判断，把不需要做的工作排除掉。

7.5 不足与改进方向

系统目前仍有若干不足，如实记录如下。第一，离线向量模型使用的是哈希向量而非真正的语义向量，它捕捉的是词汇层面的重叠，不具备同义扩展能力，因此语义检索通道的实际效果弱于使用预训练模型的方案。系统已经为此预留了接口，配置在线向量模型并重建索引即可切换。

第二，单字查询无法命中。这是 ngram 解析器按二元切分的固有限制，在界面中已给出提示，但更好的做法是在查询进入检索之前做一次判断，对单字查询自动改用其他策略。

第三，问答目前只能做单跳检索，无法处理“需要跨多个文档综合才能回答”的复杂问题。改进方向是实现多跳检索：把第一轮检索的结果作为线索发起第二轮检索，直至收集到足够的信息。

第四，本体概念的自动标注依赖词典与别名的精确匹配，对于未收录在词典中的长尾概念召回有限，后续可以考虑引入基于向量相似度的候选推荐，由人工确认后入库。

第五，多人同时编辑同一篇文档时没有冲突检测机制，后保存的内容会覆盖先保存的。改进方向是引入乐观锁：保存时比对版本号，不一致则提示用户合并。

整体而言，本次课程设计让我完整地经历了一个软件系统从需求分析、数据库设计、编码实现到测试验证的全过程，也第一次真切体会到“能跑通”与“做得好”之间的距离。系统中最有价值的部分往往不是功能本身，而是那些为了让功能真正可靠而做出的判断与取舍 —— 它们才是可以迁移到下一个项目中的经验。